Southern New Hampshire University

CS 255 Business Requirements Document

System Components and Design

Purpose

What is the purpose of this project? Who is the client and what do they want their system to be able to do?

- The client is DriverPass. Its owner, Liam, noticed that a lot of people fail the DMV driving test, and there aren't many tools that actually train students for it.
- The purpose of this project is to build a system that lets DriverPass offer online classes, online practice tests, and on-the-road driving lessons so students are better prepared to pass.
- DriverPass wants the system to let customers make, change, and cancel driving lesson reservations online, while the secretary can also do this for customers who call or walk in.
- DriverPass also wants the system to keep track of who does what (reservations, cancellations, changes), so the owner and IT officer can see who is responsible if something goes wrong.
- Our consulting company will write down what DriverPass needs (this document) and then design a system that meets those needs.

System Background

What does DriverPass want the system to do? What is the problem they want to fix? What are the different components needed for this system?

- DriverPass wants to fix a gap in the market. Many students fail the DMV driving test because they only study old tests, and DriverPass wants to give them real practice plus hands-on training.
- DriverPass wants one system, run on the web and preferably in the cloud, that handles all of this. Ian (the IT officer) said they don't want to deal with backups and security themselves.
- The owner wants to get to his data from anywhere, from any computer or mobile device, and download reports to work on in Excel at home. Data can only be changed while online, so we don't end up with duplicate data on different servers. Offline use is limited to downloaded reports.
- DriverPass has 10 cars and each car has a driver. Customers pick one of three packages. Package One is 6 hours of driving. Package Two is 8 hours of driving plus an in-person lesson on DMV rules. Package Three is 12 hours of driving, the in-person lesson, and the online class with practice tests. Each driving session is 2 hours.
- The system needs these components:
    - Customer registration and account management (including password reset).
    - Reservation and scheduling (customer, driver, car, date and time).
    - Package management (including disabling a package).
    - Online classes and practice tests, with test progress tracking.
    - User roles and access rights (owner, IT officer, secretary, customer).
    - Activity tracking and printable activity reports.
    - Report downloads for Excel.
    - A connection to the DMV so DriverPass gets notified about new rules, policies, and sample questions.
    - A web interface, plus a cloud host that handles backup and security.

Objectives and Goals

What should this system be able to do when it is completed? What measurable tasks need to be included in the system design to achieve this?

- Customers can register, log in, and make, modify, and cancel 2-hour lesson reservations online without calling the office.
- The secretary can create and change reservations and customer info for people who call or come in.
- Every reservation is matched to one customer, one driver, one car, and one time slot, and the system won't double-book a driver or a car.
- The system logs 100% of reservation changes (who made it, who canceled it, who modified it last, and when), and the owner can print an activity report from that log.
- Ian can reset or block any account from the system. Customers can reset their own password automatically.
- The owner can disable a package so no new customers can sign up for it, without needing a developer.
- Customers in Package Three can take online classes and practice tests, and the system shows each test's name, time taken, score, and status (not taken, in progress, failed, passed).
- The system gets updates from the DMV and sends a notification to DriverPass each time one comes in.
- The owner can access data online from any computer or mobile device and download reports (for example to Excel).
- The finished system is delivered and signed off by May 10 following the project schedule.

Requirements

Nonfunctional Requirements

In this section, you will detail the different nonfunctional requirements for the DriverPass system. You will need to think about the different things that the system needs to function properly.

Performance Requirements

What environments (web-based, application, etc.) does this system need to run in? How fast should the system run? How often should the system be updated?

- The system shall be web-based and run in a browser on desktop computers and mobile devices. It should be hosted in the cloud.
- Ian wants the cloud host to handle backup and security so DriverPass can focus on the business.
- Speed wasn't given in the interview, so I'm suggesting that normal pages load in about 3 seconds or less, and that saving a reservation takes about the same. This should be confirmed with Liam and Ian.
- The system should be available whenever customers want to book online. I'm suggesting high availability with maintenance done at off-hours.
- The system needs to be updated when DMV rules or sample questions change. Those content updates should be applied as soon as the DMV sends them. Software and platform updates would be scheduled by the cloud provider and Ian.

- Data can be changed only while online. Offline, users can only view reports they already downloaded.

Platform Constraints

What platforms (Windows, Unix, etc.) should the system run on? Does the back end require any tools, such as a database, to support this application?

- Since it's a web application, it should work on any operating system (Windows, macOS, Linux, iOS, Android) with a modern browser.
- The server side should run on a cloud platform. The exact OS (such as Linux or Windows Server) can be picked by the cloud provider; I'd suggest Linux.
- The back end needs a database to store customers, users, reservations, drivers, cars, packages, tests and scores, DMV updates, and the activity log. A relational database fits since the data is connected (customer to reservation to driver to car).
- The system needs a way to connect to the DMV to receive updates.
- Reports must be exportable in a format Excel can open, like CSV or XLSX.

Accuracy and Precision

How will you distinguish between different users? Is the input case-sensitive? When should the system inform the admin of a problem?

- Each user gets a unique user ID, and each user has a role (owner, IT officer, secretary, customer). The role decides what they can see and do.
- Usernames shouldn't be case-sensitive (so people don't get locked out for a capital letter), but passwords are case-sensitive.
- The system should validate input, such as required fields, phone number format, state, zip code, and credit card number and expiration date.
- The system should check that the pickup location is filled in and that the drop-off location defaults to the same address.
- Reservations must be exact (day, start and end time, driver, and car) so there are no overlaps.
- Every change to a record is saved with the user ID and a date and time stamp.
- The system should alert the admin (Ian) when something goes wrong, such as repeated failed logins or locked accounts, database or server errors, or a failed DMV update.

Adaptability

Can you make changes to the user (add/remove/modify) without changing code? How will the system adapt to platform updates? What type of access does the IT admin need?

- Ian should be able to add, remove, and modify users and change their roles from an admin screen, with no code changes.
- The owner (or Ian) can disable a package from the system with no code changes.
- Adding brand-new packages or modules would still need a developer. Sam said this can wait for a future release.
- Because it's a web app in the cloud, platform and browser updates should not need changes on DriverPass computers. The development team should retest after major updates.

- Ian needs full admin access to all accounts, so he can reset passwords and block users who leave the company.
- The design should keep the data, rules, and interface in separate layers so changes are easier in the future.

Security

What is required for the user to log in? How can you secure the connection or the data exchange between the client and the server? What should happen to the account if there is a "brute force" hacking attempt? What happens if the user forgets their password?

- Users log in with a unique username and a password. Passwords should have a minimum strength and be stored hashed, not as plain text.
- All data between the browser and server should be encrypted with HTTPS/TLS.
- Sensitive data, especially credit card info, should be encrypted and follow payment card rules (PCI DSS). The security code should not be stored after the payment is processed.
- Role-based access: each role can only see and do what it needs.
- Brute-force protection: after several failed login attempts (I suggest 5), the account is locked, Ian gets notified, and it is unlocked by Ian or after verification through a reset link.
- If a customer forgets their password, they can reset it automatically using a time-limited link sent to their email.
- Ian can block a user's access right away (for example, an employee who is let go).
- Sessions should time out after a period of inactivity.
- The cloud host handles server security and backups.

Functional Requirements

Using the information from the scenario, think about the different functions the system needs to provide. Each of your bullets should start with "The system shall . . ." For example, one functional requirement might be, "The system shall validate user credentials when logging in."

- The system shall validate user credentials when logging in.
- The system shall let a customer register with first name, last name, address, phone number, state, and credit card number, expiration date, and security code.
- The system shall store a pickup location and a drop-off location for each customer, with the drop-off defaulting to the pickup location.
- The system shall let customers choose one of the available packages (Package One, Two, or Three).
- The system shall let the owner disable a package so new customers can no longer register for it.
- The system shall let customers make, modify, and cancel driving lesson reservations online using their account.
- The system shall let the secretary make, modify, and cancel reservations for customers who call or visit the office.
- The system shall schedule each driving session as a 2-hour lesson on a day and time chosen by the customer.
- The system shall assign a driver and a car to each reservation and track which customer is matched with which driver, car, and time.

- The system shall prevent a driver or car from being booked for two sessions at the same time.
- The system shall track the hours used and remaining in each customer's package.
- The system shall give Package Three customers access to the online class and practice tests.
- The system shall show each customer's online test progress, including test name, time taken, score, and status (not taken, in progress, failed, passed).
- The system shall show driver notes, including lesson time, start hour, end hour, and driver comments.
- The system shall let drivers enter comments about each lesson.
- The system shall record who made, canceled, and last modified each reservation, with a date and time.
- The system shall generate a printable activity report so the owner can see who is responsible for changes.
- The system shall let the owner download reports in a format Excel can open.
- The system shall let Ian add, modify, and remove users and assign roles.
- The system shall let Ian reset any account's password and block any account.
- The system shall let customers reset their own forgotten password automatically.
- The system shall let users view data online from any computer or mobile device, and only allow changes to data while online.
- The system shall connect to the DMV to receive updated rules, policies, and sample questions.
- The system shall notify DriverPass when the DMV sends an update.
- The system shall provide a contact page, and a way for DriverPass to contact a student.
- The system shall alert the admin when there is a system problem or a suspected brute-force attempt.

User Interface

What are the needs of the interface? Who are the different users for this interface? What will each user need to be able to do through the interface? How will the user interact with the interface (mobile, browser, etc.)?

- The interface is a web application that works in a browser on desktop and mobile. It needs to be simple so customers, the secretary, and staff can use it with little training.
- Users: (1) Customer/student – registers, logs in, reserves, changes, and cancels lessons, takes online classes and tests, views test progress and driver notes, resets password. (2) Secretary – registers customers and manages reservations by phone or in person. (3) Owner (Liam) – views all data, activity reports, and downloads reports, and disables packages. (4) IT officer (Ian) – full access, manages users and roles, resets passwords, blocks accounts. (5) Driver – enters comments for lessons.
- The main student screen should follow Liam's sketch, with the DriverPass logo at the top and four areas: online test progress (top left), student information (top right), driver notes (bottom left), and special needs plus driver photo and student photo (bottom right).
- Online test progress shows test name, time taken, score, and status.
- Student information shows first name, last name, address, city, state, zip, phone, email, and so on.
- Driver notes show a table with lesson time, start hour, end hour, and driver comments.

- Other pages needed: login, forgot password, an input form for student information (used by the student or secretary), a reservation page, a contact page, an admin page for users and roles, a package page, and report pages.
- The interface should show clear error messages and confirm when a reservation is saved, changed, or canceled.

Assumptions

What things were not specifically addressed in your design above? What assumptions are you making in your design about the users or the technology they have?

- Customers have internet access and a modern browser or smartphone.
- Every customer has an email address so the password reset link can be sent.
- Payment will go through a secure payment service. The interview didn't say how payments or prices work, so I assume a third-party payment processor.
- The DMV can send updates electronically (like a feed or API) and will cooperate with DriverPass.
- Drivers can log in to the system and enter their own comments (the interview mentions driver comments but not how they get entered).
- DriverPass has 10 cars and 10 drivers, and these numbers won't change much at launch.
- Business hours and which time slots are available were not given, so I assume the office will set them.
- The in-person lessons for Packages Two and Three are scheduled in some way by the office, but the interview didn't cover how.
- The customer will approve the design without asking for major changes (the schedule assumes this).
- The team members (Toni, Clark, and John) are available when the schedule says they are.
- DriverPass uses one DMV and one state for now.

Limitations

Any system you build will naturally have limitations. What limitations do you see in your system design? What limitations do you have as far as resources, time, budget, or technology?

- Data can only be changed while online, so offline users can only look at downloaded reports.
- New packages or modules can't be added by a non-developer. Only disabling a package is included in this version.
- The system depends on the cloud provider for uptime, backups, and server security.
- The DMV connection depends on what the DMV offers, so we may not be able to get automatic updates if they don't have a feed.
- The schedule is tight (about 3.5 months from kickoff to sign-off) and assumes no customer change requests and no delays. The business logic step is the longest at 22 days.
- The budget wasn't discussed, so costs for cloud hosting, payment processing, and security compliance are unknown.
- Liam didn't want future features yet, so none are included.
- Only one team will work on this, so a delay in one task (like the class diagram waiting for John to return from vacation) can push back the schedule.

Gantt Chart

Please include a screenshot of the GANTT chart that you created with Lucidchart. Be sure to check that it meets the plan described by the characters in the interview.

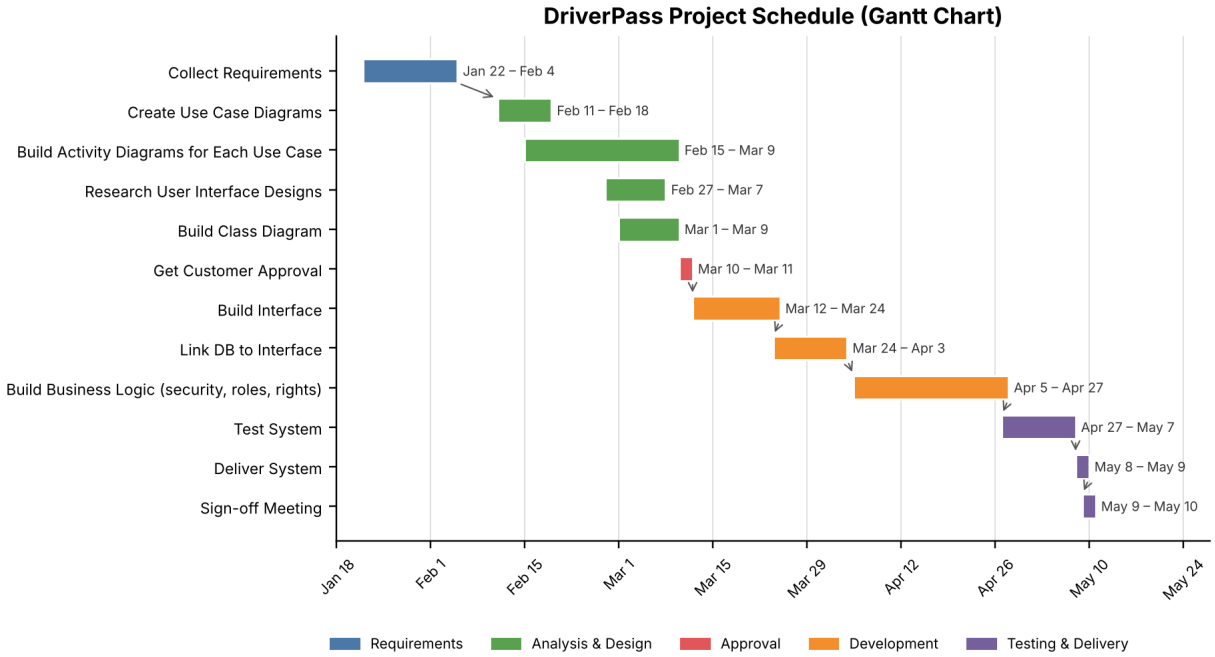


Figure 1. Project schedule based on the Sam and Jennifer schedule conversation. Each phase has its own color. Tasks run in order from top to bottom: the use case, activity, UI research, and class diagram work must finish before customer approval (Mar 10–11), and development only starts after approval.